File permissions in Linux

Project description

My assignment was to audit and correct the file and directory permissions for a research team's Linux environment. The objective was to ensure all access rights align with our organization's security policies, thereby hardening the file system against unauthorized access.

Check file and directory details

ls -la /home/researcher2/projects

Describe the permissions string

The permissions string is a 10-character code that shows the type of file and the access rights for different users.
The first character tells you the file type: a dash (-) means it’s a regular file, d indicates a directory, and l stands for a symbolic link.
The next three characters, positions 2–4 show what the owner can do, read (r), write (w), or execute (x).
Characters 5 to 7 indicate the group’s permissions.
The last three characters, 8–10 represent the permissions for all other users, also using r, w, and x.
For example, -rw-r--r-- indicates a regular file where the owner can read and write, while the group and others can only read it.

Change file permissions

chmod o-w /home/researcher2/projects/project_m.txt

The write (w) permission for the "other" (o) user group is removed (-) by this command. This was required since the file's initial permissions created a security concern by enabling anyone on the system to alter it. This modification guarantees that the file can only be written to by the owner and group members.

Change file permissions on a hidden file

chmod o-w /home/researcher2/projects/.project_x.txt

Like the file above, it was discovered that "other" (o) users may write to this hidden project file. In order to protect the archived file from unwanted alteration, this command eliminates (-) that write (w) permission.

Change directory permissions

chmod 700 /home/researcher2/projects/drafts

The notation used in this command is octal (numeric). Rwx------ is the translation of the number 700. This action was made to limit access to the drafts directory to its owner only, which is a typical security practice. It grants the user (owner) full read, write, and execute permissions while deleting all permissions from the group and "other."

Summary

In this task, I successfully hardened the security of a Linux file system. I began by auditing existing permissions with ls -la to find configurations that did not comply with security policies. I then used the chmod command with both symbolic and octal methods to remediate these issues. The result is a more secure environment that properly enforces the principle of least privilege by removing unnecessary public write access and restricting directory access to only the owner.